Parcial 3, Parte 2: de RTL a Physical Design con Synopsys

Arquitectura de Computadoras, Universidad del Istmo de Guatemala

José Rueda, carné 18123

Resumen

Esta versión de la Parte 2 está hecha con las herramientas que pide el parcial: VCS y Verdi para simular, Design Compiler (DC) para sintetizar e IC Compiler II (ICC2) para el diseño físico, y termina con archivos GDSII, DEF y netlist. Reemplaza al informe anterior hecho en Vivado, que entregué primero porque el servidor de Synopsys estaba caído el día de la entrega. El informe de Vivado sigue en el repositorio como respaldo.

Siguiendo la instrucción de la Parte 2, el diseño usa un pipeline para dividir la división en varias etapas y lograr paralelismo, y trabaja con un reloj de 65 ns, más largo que los 20 ns que tenía antes.

Etapa	**Herramienta**	**Resultado**
Simulación RTL de alu_sync	VCS + Verdi	0 errores (5,000 vectores)
Simulación RTL de alu_sync_div	VCS + Verdi	0 errores (20,000 vectores, latencia 6)
Síntesis, reloj de 65 ns	DC shell	Timing cumplido (en 60 ns: -0.04 ns; en 25 ns: -30.3 ns)
Simulación post-síntesis (netlist de DC)	VCS	0 errores
Floorplan, alimentación, placement, pines, ruteo, fillers	ICC2	0 DRCs, 0 nets abiertas
Exportación	ICC2	DEF, netlist y GDSII (el GDSII sin dibujo interno de celdas, ver sección 7)
CTS y timing post-ruteo	ICC2	No se pudo: faltan los archivos de parásitos (TLU+), ver sección 7

1. Preparación y simulación del RTL

La base es la ALU+ de 16 bits de la Parte 1 (alu.sv, sin cambios): sumador Kogge-Stone (prefix adder) porque mi carné es impar, multiplicador porque mi nombre termina en vocal, AND y OR con buffers de tres estados, y las banderas N, Z, C, V. Los códigos de alu_control son: 000 suma, 001 resta, 010 AND, 011 OR, 100 multiplicación, 101 cociente, 110 residuo y 111 cero.

Para la Parte 2 la ALU se vuelve síncrona: alu_sync.sv le pone registros a las entradas (a, b y alu_control) y a las salidas (result y las banderas). El reset es asíncrono y activo en alto. Con esos dos juegos de registros, el resultado sale 2 ciclos después de cambiar las entradas.

El testbench (tb_alu_sync.sv) usa un reloj de 10 ns, 5,000 vectores aleatorios, un 25 % de ellos con a = b para probar la bandera Z, y un modelo de referencia escrito aparte que tiene en cuenta el retraso de 2 ciclos. Se compila y se corre con:

```
vcs -full64 -sverilog -kdb -debug_access+all -f files.f -o simv
./simv
verdi -dbdir simv.daidir -ssf dump.fsdb &
```

Resultado: Errores: 0. En Verdi agregué las señales a, b, alu_control, reset y result a la forma de onda.

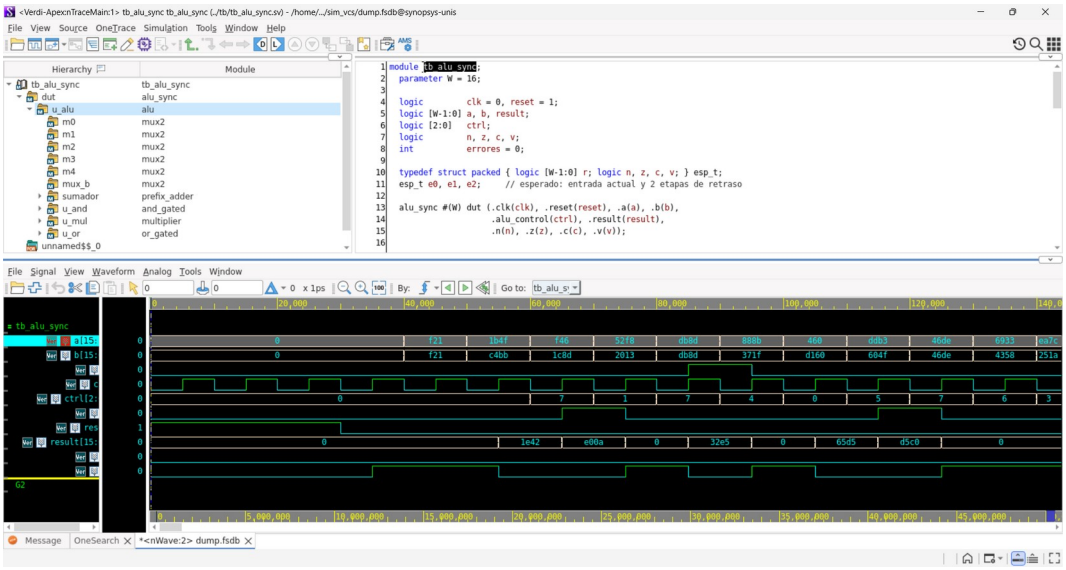


Figura 1. Forma de onda en Verdi (nWave) de la simulación de alu_sync: entradas a, b, alu_control, reset y result.

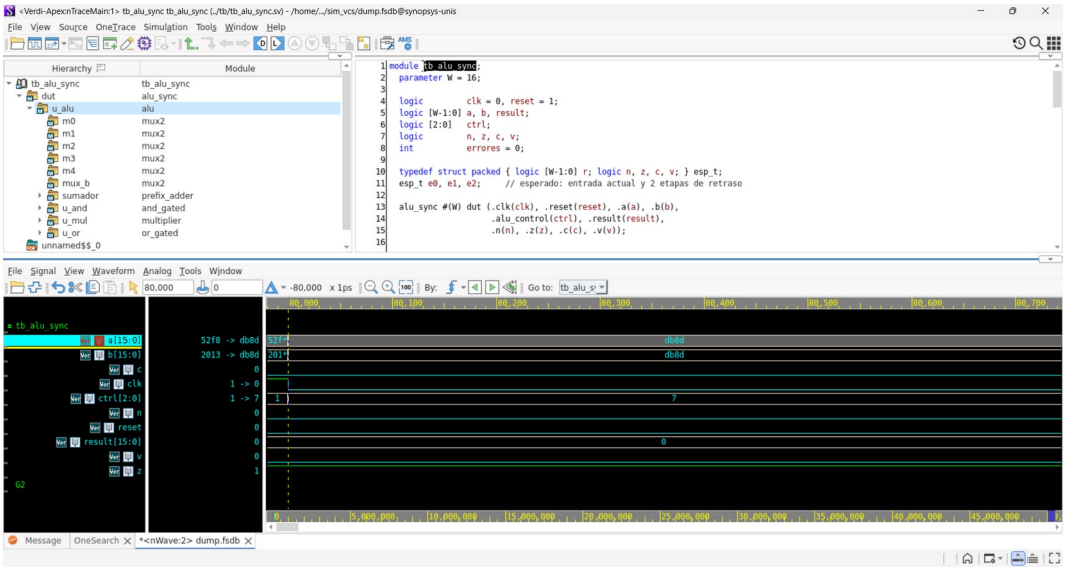


Figura 2. La misma simulación con el cursor en una operación, que muestra los valores de a, b y result en ese instante.

2. El divisor con pipeline

La instrucción de esta parte pide usar un pipeline para dividir la división en distintas partes y conseguir cierto paralelismo, y usar otro reloj con un período mayor a los 20 ns que tenía. La división es la operación más lenta de la ALU, porque en una sola etapa necesita 16 iteraciones encadenadas de resta y comparación.

Por eso escribí divider_pipe.sv, un divisor con restauración (restoring) de W bits dividido en STAGES etapas. Cada etapa hace W / STAGES iteraciones (con W = 16 y STAGES = 4, son 4 iteraciones por etapa) y entre una etapa y la siguiente hay registros que guardan el dividendo restante, el residuo, el cociente parcial y el divisor. Así, en cada ciclo entra una división nueva mientras las anteriores avanzan por las otras etapas, que es el paralelismo del pipeline.

La división entre cero devuelve el cociente con todos los bits en 1 y el residuo igual al dividendo (la convención de RISC-V), para que el resultado esté definido.

alu_sync_div.sv une la ALU de la Parte 1 con este divisor. Las operaciones que no son de división se retrasan con una línea de retardo (delay_line) de STAGES ciclos, para que todas las operaciones tengan la misma latencia total: STAGES + 2 = 6 ciclos. Así el resultado siempre sale en el orden en que entraron los datos. Para el cociente (101) y el residuo (110), las banderas N y Z se recalculan a partir del resultado y C y V valen 0.

El testbench tb_alu_sync_div.sv usa 20,000 vectores aleatorios, con casos extra de a = b, divisores pequeños, divisor cero y más divisiones, comparados contra un modelo de referencia con la misma latencia. Resultado: Errores: 0 (W=16, STAGES=4, LAT=6).

3. Síntesis con Design Compiler

El script dc_alu_sync_div.tcl usa la librería M31LLSC750GL150MS (esquina 25 °C, TT, 1.8 V) y las librerías sintéticas de DesignWare. Las restricciones son: reloj en el puerto clk, incertidumbre del 10 % del período, retardos de entrada de 20 % del período (máximo) y 5 % (mínimo), retardos de salida de 20 % (máximo) y carga de salida de 0.04. La síntesis se hace con compile_ultra y sin deshacer la jerarquía, para ver el área por módulo.

Dos observaciones que cambiaron el resultado:

- En la primera corrida, el camino más lento empezaba en el puerto reset, pasaba por el pin RN del flip-flop y seguía por la lógica. El reset es asíncrono, así que no es un camino de datos que deba cumplir el período. Lo corregí con set_false_path -from reset y quité reset de las entradas con retardo.
- Con esa corrección quedó el camino real: la ruta de la ALU, sobre todo el multiplicador, de unos 51.5 ns. Como la incertidumbre es el 10 % del período, el reloj tiene que ser más largo que ese camino más ese margen.

Probé tres períodos:

Período del reloj	**Peor slack (WNS)**	**Resultado**
25 ns	-30.3 ns	No cumple
60 ns	-0.04 ns	No cumple por muy poco
65 ns	≥ 0	Cumple

Por eso entrego la síntesis con 65 ns. Queda un aviso de max_leakage_power, que viene de un objetivo por defecto de 0 que DC trae y que no definí; no es una violación de timing y lo dejo documentado. El área, la potencia y la cantidad de celdas están en los reportes de reports_dc_65ns (area.rpt, power.rpt, qor.rpt).

Salidas de DC: alu_sync_div_dc.v (netlist), alu_sync_div_dc.sdc y alu_sync_div_dc.ddc.

4. Simulación post-síntesis

Para comprobar que lo que sintetizó DC se comporta igual que el RTL, compilé el mismo testbench de alu_sync_div pero con el netlist de DC y el modelo Verilog de las celdas del kit en lugar del RTL. Con +nospecify y +notimingchecks, la simulación es funcional y no usa los retardos de las celdas.

```
vcs -full64 -sverilog -f files_gl.f -o simv_gl
./simv_gl | grep Errores
# Errores: 0  (W=16, STAGES=4, LAT=6)
```

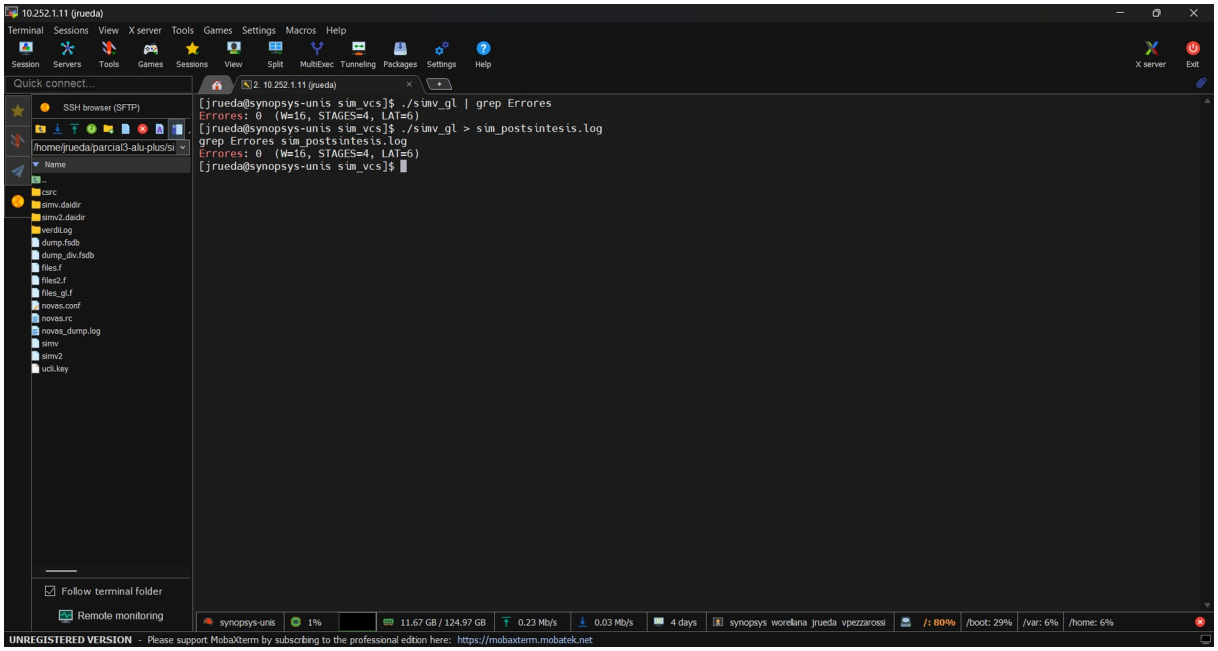


Figura 3. Simulación del netlist sintetizado: 0 errores con los 20,000 vectores.

VCS muestra avisos TMPO (el testbench pide parámetros que el netlist ya trae fijos en 16 bits y 4 etapas, y los ignora, con los mismos valores) y TFIPC (DC dejó sin conectar las salidas que no se usan de los sumadores del multiplicador). Ninguno afecta el resultado.

5. Diseño físico con ICC2

ICC2 necesita las celdas del kit en su formato propio (NDM). Con icc2_lm_shell armé la librería M31LLSC750GL150MS.ndm a partir del LEF, el archivo tecnológico y los modelos de celdas (build_ndm.tcl). Solo aparecieron avisos por la dirección de los pines de alimentación.

El flujo de alu_icc2.tcl y alu_icc2_fisico.tcl es:

- Leer el netlist de DC (alu_sync_div) y el SDC, y crear las redes de alimentación VCCK (positiva) y GNDK (tierra), que son los nombres de este kit, no VDD y VSS.
- Floorplan con una utilización del núcleo de 50.01 %, relación 1:1 y un margen de 10 alrededor.
- Malla de alimentación: METAL5 horizontal y METAL4 vertical, con ancho 2 y paso 30, más rieles en METAL1 para las celdas.
- Colocación de celdas (create_placement), legalización (legalize_placement) y colocación de pines.
- Ruteo (route_auto), inserción de fillers y conexión de alimentación a todas las celdas.

Resultados verificados con check_routes y check_lvs: 3,645 nets, 0 nets abiertas y 0 DRCs. Al insertar los fillers aparecieron dos violaciones (un corto en METAL1 y una violación de espaciado entre nets distintas) que corregí con remove_stdcell_fillers_with_violation, que eliminó esos dos fillers. Después de eso, el reporte final (final_v2) quedó en 0 DRCs.

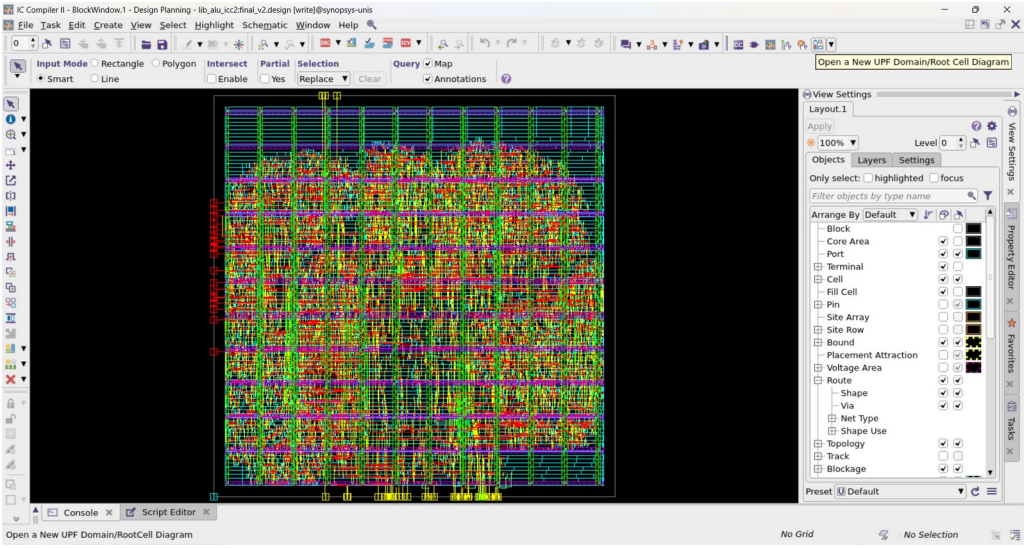


Figura 4. Layout final de alu_sync_div en la interfaz de ICC2: el borde del diseño, el área de celdas con su ruteo y los pines en los bordes.

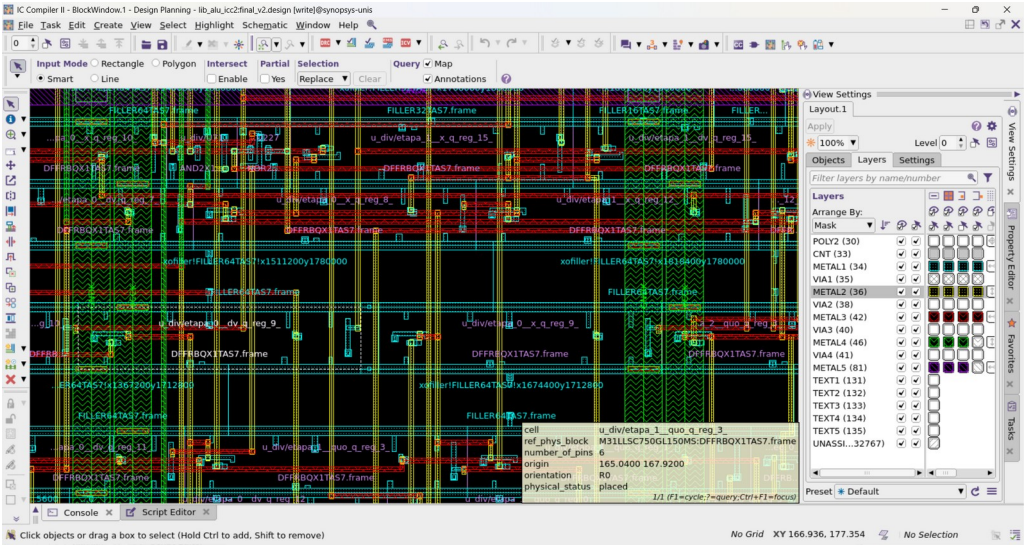


Figura 5. Zoom a una zona: se ven celdas colocadas, como un flip-flop de la etapa 1 del divisor (u_div/etapa_1__quo_q_reg_3_, tipo DFFRBQX1TAS7, estado placed), junto con los fillers y el ruteo.

6. Cómo leer los colores del layout

Cada capa de metal tiene su propio color en ICC2, por eso la vista parece tan cargada: es normal, son todas las capas del chip dibujadas una encima de otra. Según el panel de capas de ICC2:

Capa	Color en ICC2	Qué contiene
METAL1	azul / cian	Conexiones locales entre celdas y rieles de alimentación de las filas
METAL2	amarillo	Ruteo de señales
METAL3	rojo	Ruteo de señales
METAL4	verde	Malla de alimentación vertical, además de ruteo
METAL5	morado	Malla de alimentación horizontal (bandas gruesas)

Los cuadros pequeños en los bordes (rojos a la izquierda, amarillos arriba y abajo) son los pines de entrada y salida del diseño. Desmarcando capas en el panel Layers se puede ver cada una por separado. La alimentación del chip (VCCK y GNDK) está en la malla de METAL4 y METAL5, por eso el diseño no necesita una fuente de voltaje ni resistencias como las de un circuito armado con componentes: las celdas reciben alimentación de esa malla, y la tensión nominal (1.8 V) viene de la librería.

7. Limitaciones

- Parásitos: el kit M31 que me dieron no trae archivos TLU+ ni ITF (los modelos de resistencia y capacitancia de los cables). Sin ellos, place_opt, clock_opt (la síntesis del árbol de reloj, CTS) y route_opt terminan con el error "No corner has valid parasitic". Por eso corrí el flujo sin las etapas que optimizan por timing: placement y ruteo sin CTS y sin el análisis de timing después del ruteo. Los archivos para hacerlo tienen que venir del catedrático (TLU+ o ITF con su archivo de mapeo para M31). Con ellos se podrían correr esos pasos y obtener el timing después del ruteo.
- GDSII: el archivo se escribe, pero ICC2 avisa (GDS-034) que no tiene la vista de layout de cada celda, porque el kit no trae el GDS de las celdas. Por eso el GDSII contiene la estructura del chip (ruteo, mallas, pines y posiciones de las celdas), pero no el dibujo interno de cada compuerta.
- Timing: el cumplimiento de timing está verificado en DC (65 ns), no después del ruteo.

8. Archivos entregados

Archivo o carpeta	**Contenido**
rtl/, tb/	Código de la Parte 1 y de la Parte 2, incluidos divider_pipe, alu_sync_div y sus testbenches
dc_alu_sync_div.tcl	Script de síntesis
build_ndm.tcl, alu_icc2.tcl, alu_icc2_fisico.tcl, fix_fillers.tcl	Scripts de ICC2
reports_dc_65ns/, run_dc_65ns/	Reportes y salidas (netlist, SDC, DDC) de DC a 65 ns
reports_icc2/, out_icc2/	Reportes de ICC2 y salidas: DEF, netlist y GDSII
sim_postsintesis.log, logs de DC e ICC2	Salida de las corridas
capturas_parte2_synopsys/	Capturas de Verdi, de la simulación post-síntesis y del layout

Referencia: la versión anterior de la Parte 2 con Vivado (informe, reportes y netlists) sigue en las carpetas informe/, reportes_parte2/ y capturas_parte2/ del repositorio.